

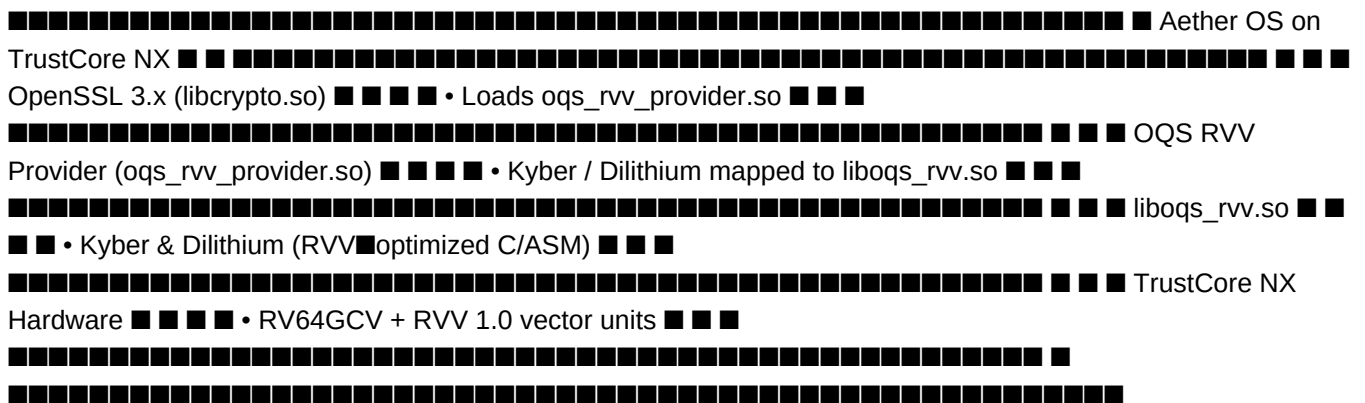
Aether One™ – PQC RVV Integration Guide (Kyber & Dilithium)

Compiling Kyber and Dilithium with the RISC-V RVV toolchain and integrating them into Aether OS via OpenSSL providers.

Purpose

This guide describes how to build RVV-accelerated implementations of Kyber and Dilithium using the GNU RISC-V toolchain and how to integrate them into Aether OS as OpenSSL 3 providers. The goal is to enable TrustCore NX to execute post-quantum key encapsulation and digital signatures using the same RVV resources that accelerate AI workloads.

System Architecture



1. RISC-V RVV Toolchain Configuration

Assuming the RVV-enabled GNU toolchain is already built as described in the RVV 1.0 Toolchain Guide:
export RISCVC_TOOLCHAIN=\$HOME/riscv/toolchain/install export PATH=\$RISCVC_TOOLCHAIN/bin:\$PATH
export CC=riscv64-unknown-linux-gnu-gcc export CFLAGS="-O3 -march=rv64gcv -mabi=lp64d -mattr=+v,+zfh"
These settings ensure that all PQC C code is compiled for RV64GCV with vector and half-precision support.

2. Building liboqs with RVV Support (Kyber & Dilithium)

Clone and build the Open Quantum Safe library with the RVV-enabled compiler:
git clone https://github.com/open-quantum-safe/liboqs.git cd liboqs mkdir build && cd build cmake .. -GNinja \-DCMAKE_SYSTEM_NAME=Linux \-DCMAKE_C_COMPILER=\$CC \-DOQS_DIST_BUILD=OFF \-DOQS_USE_OPENSSL=ON \-DCMAKE_C_FLAGS="\$CFLAGS" \-DCMAKE_INSTALL_PREFIX=/usr/local/trustcore_rvv ninja ninja install This produces a RVV-aware liboqs.so that includes Kyber and Dilithium implementations compiled for RV64GCV.

3. RVV-Optimized Kernel Example (Kyber Polynomial Multiply)

```
#include <riscv_vector.h>
#include <stddef.h>
#include <stdint.h>

void poly_mul_rvv(uint16_t *r, const uint16_t *a, const uint16_t *b, size_t n) {
    for (size_t i = 0; i < n; i += vsetvl_e16m1(n - i)) {
        size_t vl = vsetvl_e16m1(n - i);
        vint16m1_t va = vle16_v_i16m1(&a[i], vl);
        vint16m1_t vb = vle16_v_i16m1(&b[i], vl);
        vint16m1_t vr = vmul_vv_i16m1(va, vb, vl);
        vse16_v_i16m1(&r[i], vr, vl);
    }
}
```

```
}
```

This function can replace scalar polynomial multiplication loops in Kyber's NTT pipeline, leveraging RVV 1.0.

4. OpenSSL 3 Provider Integration (Aether OS)

OpenSSL 3.x supports algorithm providers that allow Kyber and Dilithium to be exposed via standard EVP APIs. An `oqs_rvv_provider.so` module links against the RVV-enabled compiled `liboqs.so` and registers PQC mechanisms.

Example provider declaration snippet:

```
static const OSSL_ALGORITHM oqs_rvv_kems[] = { { "KYBER768:riscv_rvv", "provider=oqs_rvv",  
oqs_kyber_rvv_functions }, { "DILITHIUM3:riscv_rvv", "provider=oqs_rvv", oqs_dilithium_rvv_functions }, { NULL,  
NULL, NULL } };
```

At runtime, the provider is loaded as:

```
OSSL_PROVIDER *p = OSSL_PROVIDER_load(NULL, "oqs_rvv");
```

5. Testing PQC on Aether OS

Once deployed on TrustCore NX with Aether OS:

```
openssl list -providers
```

The output should list `oqs_rvv` and its algorithms.

Test Kyber key generation and Dilithium signing:

```
openssl genpkey -algorithm KYBER768:riscv_rvv -out kyber_key.pem openssl pkey -in kyber_key.pem -text  
openssl genpkey -algorithm DILITHIUM3:riscv_rvv -out dilithium_key.pem openssl dgst -sign dilithium_key.pem  
-out sig.bin testfile.txt
```

If these commands succeed, the RVV-enabled accelerated PQC path is correctly integrated.

Summary

By compiling Kyber and Dilithium with the RVV-enabled GNU RISC-V toolchain and exposing them through an OpenSSL 3 provider, Aether OS gains hardware-accelerated, post-quantum-secure key exchange and signature capabilities on TrustCore NX. This unifies AI and cryptography acceleration on the same RVV 1.0 vector engine within the Aether One prototype stack.